Dockside Technical Developer Documentation

This document serves as a comprehensive technical reference for the Dockside TypeScript API client and its associated Vue 3 composables. It is designed for developers integrating Dockside capabilities into their applications, providing detailed insights into the architecture, available methods, reactive state management, and type definitions.

API Client Architecture

The Dockside API client is built upon a modular architecture, utilizing a foundational `BaseClient` that handles core HTTP request logic, error parsing, and query string construction. Specialized client classes extend this base to provide domain-specific methods for interacting with various Docker and Dockside services. The primary entry point for developers is the `DocksideClient` class, which aggregates these specialized clients into a single, cohesive instance.

The `BaseClient` requires a base URL and a `fetch` implementation upon instantiation. It provides a protected `request<T>` method that standardizes API calls, ensuring consistent headers (like `Content-Type: application/json`) and uniform error handling by throwing standard JavaScript `Error` objects when HTTP responses indicate failure.

Specialized API Clients

The following table details the specialized clients available within the `DocksideClient` instance, outlining their primary responsibilities and key methods.

Client Property	Class Name	Description	Key Methods
`workspace`	`WorkspaceClient`	Manages Dockside workspace state, projects, and Docker Compose configurations.	`getState`, `createProject`, `toggleFavorite`, `updateStatus`, `getCompose`, `saveCompose`
`containers`	`ContainersClient`	Facilitates interaction with Docker containers, including lifecycle management and inspection.	`list`, `inspect`, `remove`, `start`, `stop`, `restart`, `logs`
`images`	`ImagesClient`	Handles operations related to Docker images.	`list`, `inspect`, `remove`
`system`	`SystemClient`	Retrieves Docker system information, version details, and performs system pruning.	`version`, `info`, `descriptor`, `prune`
`orchestration`	`OrchestrationClient`	Manages Docker orchestration tasks, such as deploying projects and listing networks/volumes.	`deploy`, `getComposeLink`, `listNetworks`, `listVolumes`
`registry`	`RegistryClient`	Interacts with Docker registries to fetch image manifests, metadata, and tags.	`manifest`, `metadata`, `info`, `tags`, `update`

Vue 3 Composables Reference

The provided Vue 3 composables abstract the complexity of API interactions and state management, offering reactive interfaces tailored for Vue components. These composables leverage the underlying

`docksideClient` to fetch data and perform actions, while managing loading states, error handling, and toast notifications automatically.

Core State Management Composables

These composables are responsible for fetching and managing data from the Dockside API. They typically return reactive references for the data, loading status, and error messages, along with methods to trigger API calls.

Composable	Purpose	Key Returns (Reactive State & Methods)
`useWorkspace()`	Manages workspace projects and state.	`state`, `loading`, `error`, `refresh()`, `createProject()`, `saveCompose()`
`useContainers()`	Manages the list of Docker containers and their lifecycle.	`containers`, `loading`, `error`, `refresh()`, `start()`, `stop()`, `restart()`, `remove()`
`useImages()`	Manages the list of Docker images.	`images`, `loading`, `error`, `refresh()`, `remove()`
`useSystem()`	Retrieves Docker system information.	`version`, `info`, `descriptor`, `loading`, `error`, `refresh()`, `prune()`
`useOrchestration()`	Manages Docker networks, volumes, and project deployment.	`networks`, `volumes`, `loading`, `error`, `refresh()`, `deploy()`
`useRegistry()`	Fetches information from Docker registries.	`tags`, `metadata`, `manifest`, `update`, `loading`, `error`, `load()`

Configuration and Normalization Composables

These composables assist in managing and normalizing complex configuration objects, particularly Docker Compose definitions.

Composable	Purpose	Key Returns
`useComposeConfig(initial?)`	Manages a reactive Docker Compose configuration.	`name`, `version`, `services`, `networks`, `volumes`, `config` (computed)
`useService(initial)`	Manages the reactive state of a single Docker service.	`state`, `update()`, `image`, `environment`, `ports`
`useComposeConfigNormalizer()`	Normalizes raw objects into structured `ComposeConfig` types.	`normalizeComposeConfig()`, `normalizeServices()`
`useServiceNormalizer()`	Normalizes raw service definitions into structured `Service` types.	`normalizeService()`, `normalizePort()`, `normalizeVolume()`
`useYamlNormalizer()`	Normalizes raw YAML parsed objects into `ComposeConfig`.	`normalizeYamlObject()`

Utility and UI Composables

These composables provide general utilities and manage UI-related state, such as notifications and themes.

Composable	Purpose	Key Returns
`useDockside()`	A wrapper around `docksideClient` providing global loading/error state.	`loading`, `error`, `wrap()`, access to all API clients
`useToasts()`	Manages a global list of toast notifications.	`toasts`, `show()`, `success()`, `error()`, `warning()`, `info()`
`useNotifications()`	Manages application notifications and panel states.	`notifications`, `flyoutOpen`, `centerOpen`, `pushNotification()`, `toggleFlyout()`
`useTheme()`	Manages the application theme (light/dark/system).	`theme`, `applyTheme()`
`useWindowSize()`	Provides reactive window dimensions and desktop breakpoint detection.	`width`, `height`, `isDesktop`
`useCommands()`	Manages a global registry of executable commands.	`commands`, `registerCommand()`, `unregisterCommand()`
`useCapabilityDescriptions()`	Provides descriptions for Linux capabilities.	`descriptions`, `allCapabilities`, `getDescription()`, `entries`

Type Definitions

The Dockside SDK relies heavily on TypeScript interfaces to ensure type safety across API responses and configuration objects. Below are the core type definitions used throughout the system.

Docker Entity Types

These interfaces define the structure of objects returned by the Docker API endpoints.

```ts
export interface DockerContainerSummary {
  Id: string;
  Names: string[];
  Image: string;
  State: string;
  Status: string;
  Ports: DockerPortBinding[];
}

export interface DockerPortBinding {
  IP?: string;
  PrivatePort: number;
  PublicPort?: number;
  Type: string;
}

export interface DockerImageSummary {
  Id: string;
  RepoTags?: string[];
  RepoDigests?: string[];
  Created: number;
  Size: number;
  VirtualSize: number;
}

export interface DockerNetwork {
  Name: string;
  Id: string;
  Created: string;
  Scope: string;
  Driver: string;
  EnableIPv6: boolean;
  IPAM: any;
}

export interface DockerVolume {
  Name: string;
  Driver: string;
  Mountpoint: string;
  Labels?: Record<string, string>;
  Scope: string;
}
```

Workspace and Project Types

These interfaces define the state and configuration of Dockside workspaces and projects.

```
export type WorkspaceStatus = "unknown" | "running" | "syncing" | "stopped" |

export interface WorkspaceProjectState {
  projectName: string;
  configPath: string;
  folderPath: string;
  dsId: string;
  status: WorkspaceStatus;
  isFavorite: boolean;
  services: WorkspaceService[];
}

export interface WorkspaceService {
  name: string;
  image: string;
  state: "running" | "stopped" | "starting" | "error" | "unknown";
  health: "healthy" | "unhealthy" | "starting" | "none";
}

export interface WorkspaceState {
  rootPath: string | null;
  isLoading: boolean;
  error: string | null;
  projects: Record<string, WorkspaceProjectState>;
  lastSyncedAt: string | null;
}
```

Docker Compose Configuration Types

These interfaces represent the structured format of a Docker Compose file, used extensively by the configuration composables.

```typescript
export interface ComposeConfig {
  name?: string;
  version?: string;
  services?: Record<string, Service>;
  networks?: Record<string, Network | null>;
  volumes?: Record<string, Volume | null>;
  secrets?: Record<string, Secret>;
  configs?: Record<string, Config>;
}

export interface Service {
  name: string;
  image?: string;
  build?: string | ServiceBuild;
  command?: string | string[];
  container_name?: string;
  environment?: EnvironmentRecord | string[];
  ports?: (string | ServicePort)[];
  volumes?: (string | ServiceVolume)[];
  networks?: string[] | Record<string, ServiceNetwork | null>;
  depends_on?: string[] | Record<string, ServiceDependency>;
  restart?: "no" | "always" | "on-failure" | "unless-stopped";
  // ... numerous other service configuration properties
}

export interface ServicePort {
  mode?: "host" | "ingress";
  host_ip?: string;
  target?: number;
  published?: string | number;
  protocol?: "tcp" | "udp" | "sctp";
  description?: string;
}

export interface ServiceVolume {
  type: "bind" | "volume" | "tmpfs" | "npipe" | "cluster";
  source?: string;
  target: string;
  read_only?: boolean;
  // ... specific volume type options
}
```

Usage Examples

The following examples demonstrate how to integrate the Dockside API client and composables within a Vue 3 application.

Example 1: Managing Containers

This example illustrates how to use the `useContainers` composable to fetch a list of containers and provide functionality to start or stop them. The composable automatically handles loading states and error notifications.

```
<template>
  <div>
    <h2>Docker Containers</h2>
    <div v-if="loading">Loading containers...</div>
    <div v-else-if="error" class="error">{{ error }}</div>
    <ul v-else>
      <li v-for="container in containers" :key="container.Id">
        <strong>{{ container.Names[0] }}</strong> ({{ container.State }})
        <button @click="handleStart(container.Id)" :disabled="container.State ===
        <button @click="handleStop(container.Id)" :disabled="container.State !==
      </li>
    </ul>
    <button @click="refresh()">Refresh List</button>
  </div>
</template>

<script setup lang="ts">
import { onMounted } from 'vue';
import { useContainers } from '@/composables/useContainers';

const { containers, loading, error, refresh, start, stop } = useContainers();

onMounted(() => {
  refresh({ all: true });
});

const handleStart = async (id: string) => {
  await start(id);
  refresh({ all: true });
};

const handleStop = async (id: string) => {
  await stop(id);
  refresh({ all: true });
};
</script>
```

Example 2: Editing a Docker Compose Service

This example demonstrates the use of the `useService` composable to create a reactive editor for a specific Docker service configuration. Changes made to the reactive properties (`image`, `environment`) are automatically reflected in the underlying state.

```
<template>
  <div class="service-editor">
    <h3>Edit Service: {{ state.name }}</h3>

    <label>
      Image:
      <input v-model="image" type="text" placeholder="e.g., nginx:latest" />
    </label>

    <div class="environment-vars">
      <h4>Environment Variables</h4>
      <div v-for="(val, key) in environment" :key="key">
        <input :value="key" readonly /> = <input v-model="environment[key]" />
        <button @click="removeEnv(key)">Remove</button>
      </div>
      <button @click="addEnv">Add Variable</button>
    </div>
  </div>
</template>

<script setup lang="ts">
import { useService } from '@/composables/useService';
import type { Service } from '@/types';

const props = defineProps<{ initialService: Service }>();

const { state, image, environment } = useService(props.initialService);

const addEnv = () => {
  // Ensure environment is an object before adding
  if (Array.isArray(environment.value)) {
      // Handle conversion if necessary, or assume it's normalized to a Record
  } else {
      environment.value = { ...environment.value, 'NEW_VAR': 'value' };
  }
};

const removeEnv = (key: string | number) => {
    const newEnv = { ...environment.value };
    delete newEnv[key];
    environment.value = newEnv;
};
</script>
```

Example 3: Global API Calls with `useDockside`

For scenarios where you need to make API calls outside the context of a specific resource manager (like `useContainers`), the `useDockside` composable provides a convenient `wrap` function. This function automatically manages a global loading state and catches errors, making it ideal for custom operations.

```
<template>
  <div>
    <button @click="performSystemPrune" :disabled="loading">
      {{ loading ? 'Pruning...' : 'Prune System' }}
    </button>
    <p v-if="error" class="error-text">{{ error }}</p>
  </div>
</template>

<script setup lang="ts">
import { useDockside } from '@/composables/useDockside';
import { useToasts } from '@/composables/useToasts';

const { system, wrap, loading, error } = useDockside();
const toasts = useToasts();

const performSystemPrune = async () => {
  try {
    await wrap(() => system.prune());
    toasts.success('System pruned successfully.');
  } catch (err) {
    // Error is already captured by useDockside, but we can handle specific logic
    console.error('Prune failed', err);
  }
};
</script>
```

References

[1] Vue 3 Composition API Documentation: https://vuejs.org/guide/extras/composition-api-faq.html [2] Docker Engine API Reference: https://docs.docker.com/engine/api/ [3] Docker Compose Specification: https://docs.docker.com/compose/compose-file/